

به نام خدا



ریزپردازنده

استاد: مهدی کلباسی

گزارش پروژه

محمد صدرا خالقی

نیمسال اول ۱۴۰۵-۱۴۰۴

۱. مقدمه

این گزارش به تحلیل فنی و نظری پروژه طراحی سیستم کنترل روشنایی هوشمند مبتنی بر میکروکنترلر STM32F103 می‌پردازد. این سیستم شامل سه بخش اصلی کنترل شدت نور (Dimmer)، سیستم ایمنی مبتنی بر وقفه و گزارش‌دهی سریال است که در محیط Proteus شبیه‌سازی شده است.

۲. محاسبات دقیق تایمر (Timer Calculations)

سوال: فرمول دقیق فرکانس PWM را بنویسید و نشان دهید چه اعدادی برای PSC و ARR انتخاب کردید؟

پاسخ: برای تولید سیگنال PWM با فرکانس دقیق، از تایمر ۲ (TIM2) متصل به کلاک سیستم استفاده شده است. فرمول رابطه بین فرکانس خروجی و پارامترهای تایمر به صورت زیر است:

$$\text{Frequency} = \text{Timer_Clock} / [(\text{PSC} + 1) \times (\text{ARR} + 1)]$$

در این پروژه، کلاک سیستم (Timer Clock) برابر با ۸ MHz (معادل ۸,۰۰۰,۰۰۰ هرتز) در نظر گرفته شده است. برای دستیابی به فرکانس هدف ۱ kHz، مقادیر زیر محاسبه و انتخاب شدند:

- **Prescaler (PSC):** 79
- **Auto-Reload Register (ARR):** 99

اثبات صحت محاسبات: با جایگذاری مقادیر در فرمول بالا، فرکانس خروجی به صورت زیر محاسبه می‌شود:

$$F = 8,000,000 / [(79 + 1) \times (99 + 1)] \quad F = 8,000,000 / (80 \times 100) \quad F = 8,000,000 / 8000$$

$$F = 1000, \text{ Hz} = 1 \text{ kHz}$$

انتخاب عدد ۹۹ برای ARR باعث می‌شود تایمر از ۰ تا ۹۹ (مجموعاً ۱۰۰ پله) بشمارد. این انتخاب فرآیند نگاشت (Mapping) را ساده می‌کند، زیرا مقدار Duty Cycle مستقیماً با درصد روشنایی (۰ تا ۱۰۰) متناظر است.

۳. تحلیل معماری وقفه (Interrupt Architecture)

سوال: از لحظه‌ای که دکمه فشرده می‌شود تا لحظه‌ای که اولین خط کد تابع وقفه (Callback) اجرا می‌شود، چه اتفاقاتی در سخت‌افزار رخ می‌دهد؟

پاسخ: زمانی که دکمه متصل به پین PC13 فشرده می‌شود (لبه پایین‌رونده)، زنجیره‌ای از رخدادهای سخت‌افزاری زیر اتفاق می‌افتد:

۱. تشخیص لبه (Edge Detection): سخت‌افزار EXTI روی پین PC13 تغییر وضعیت ولتاژ را تشخیص می‌دهد.

۲. ارسال سیگنال به NVIC: کنترلر EXTI یک سیگنال درخواست وقفه به واحد مدیریت وقفه (NVIC) ارسال می‌کند.

۳. توقف CPU: اگر اولویت این وقفه بالاتر از عملیات جاری باشد، CPU اجرای برنامه اصلی را متوقف می‌کند.

۴. ذخیره کانتکست (Context Stacking): محتوای رجیسترهای مهم CPU (مانند PC, R0-R3, xPSR) به صورت خودکار در حافظه پشته (Stack) ذخیره می‌شود تا وضعیت برنامه قبلی حفظ شود.

۵. واکنشی بردار (Vector Fetch): پردازنده به جدول بردارها (Vector Table) مراجعه می‌کند تا آدرس شروع تابع وقفه (ISR) را پیدا کند.

۶. اجرای ISR: پردازنده به آن آدرس پرش کرده و کدهای داخل تابع وقفه را اجرا می‌کند.

• **Vector Table**: ناحیه‌ای از حافظه است که آدرس توابع وقفه در آنجا لیست شده است.

• **Latency (تأخیر)**: مدت زمان بین وقوع رخداد سخت‌افزاری تا اجرای اولین دستورالعمل تابع وقفه است (حدود ۱۲ سیکل کلاک در Cortex-M3).

۴. تحلیل مد GPIO و سیگنال PWM

سوال: چرا برای پین PWM از حالت Alternate Function Push-Pull استفاده می‌شود؟

پاسخ:

• **General Purpose Output**: در این حالت، وضعیت پین مستقیماً توسط CPU کنترل می‌شود. برای تولید PWM در این حالت، پردازنده باید دائماً پین را یک و صفر کند (Bit-banging) که بار پردازشی زیادی دارد و دقیق نیست.

- **Alternate Function Push-Pull:** در این حالت، کنترل پین از دست CPU خارج شده و مستقیماً به خروجی تایمر سخت‌افزاری (TIM2) وصل می‌شود. تایمر به صورت مستقل موج PWM را تولید می‌کند. این کار باعث دقت زمانی بسیار بالا و آزادسازی CPU برای کارهای دیگر می‌شود.

۵. مقایسه روش وقفه (Interrupt) با سرکشی (Polling)

سوال: چرا برای خواندن دکمه از روش Polling استفاده نکردیم؟

پاسخ:

- **روش Polling (سرکشی):** در این روش، CPU باید در حلقه اصلی دائماً وضعیت پین دکمه را چک کند. این کار باعث اتلاف ۱۰۰ درصدی توان پردازشی برای چک کردن یک پین می‌شود و اگر CPU مشغول باشد، ممکن است فشردن دکمه از دست برود.
- **روش Interrupt (وقفه):** در این روش، CPU هیچ کاری برای دکمه انجام نمی‌دهد و مشغول کارهای دیگر است. تنها زمانی که دکمه فشرده شود، سخت‌افزار به CPU اطلاع می‌دهد. در این حالت بار پردازشی برای دکمه نزدیک به صفر است و پاسخ‌دهی آنی (Real-time) می‌باشد.